🚀 Подробный разбор всех этапов DevOps проекта

Автор: Vitalii Zaburdaiev
Проект: Health Dashboard
Курс: DevOpsUA6

📚 Содержание

1. Приложение (Flask Application)
2. Docker Compose
3. CI/CD (GitHub Actions)
4. Terraform (Infrastructure as Code)
5. Ansible (Configuration Management)
6. Kubernetes (Container Orchestration)
7. Monitoring (Prometheus + Grafana)
8. Testing (Unit Tests)
9. Nginx (Reverse Proxy)

---

1. Приложение (Flask Application)

📁 Файл: `app/app.py` (13 KB)

🎯 Что это?

Это **Flask приложение** - веб-сервис для мониторинга здоровья системы. Он собирает метрики (CPU, память, диск) и предоставляет их через REST API.

🏗️ Из чего состоит приложение:

```
╔═══════════════════════════════════════════════════════════════╗
║                    FLASK APPLICATION STRUCTURE                ╒
╟═══════════════════════════════════════════════════════════════╢
│                                                               │
│  1. IMPORTS & DEPENDENCIES                                    │
│     ├── Flask (web framework)                                 │
│     ├── psutil (system metrics: CPU, RAM, disk)               │
│     ├── psycopg2 (PostgreSQL driver)                          │
│     ├── redis (Redis client)                                  │
│     └── prometheus_client (metrics export)                    │
│                                                               │
│  2. LOGGING (JSON format для Loki)                            │
│     └── Structured logs: timestamp, level, message            │
│                                                               │
│  3. PROMETHEUS METRICS                                        │
│     ├── REQUEST_COUNT (счётчик запросов)                      │
│     ├── REQUEST_LATENCY (время ответа)                        │
│     ├── SYSTEM_CPU (загрузка CPU)                             │
│     ├── SYSTEM_MEMORY (использование памяти)                  │
│     └── SYSTEM_DISK (использование диска)                     │
│                                                               │
│  4. DATABASE FUNCTIONS                                        │
│     ├── get_db_connection() - подключение к PostgreSQL        │
│     ├── init_db() - создание таблицы metrics                  │
│     └── save_metrics() - сохранение метрик в БД               │
│                                                               │
│  5. CACHE FUNCTIONS (Redis)                                   │
│     ├── get_cached_metrics() - чтение из кэша                 │
│     └── set_cached_metrics() - запись в кэш (TTL 10 сек)      │
│                                                               │
│  6. SYSTEM METRICS COLLECTION                                 │
│     └── collect_system_metrics() - собирает CPU/RAM/Disk      │
│                                                               │
│  7. FLASK ROUTES (API Endpoints)                              │
│     ├── GET /              - HTML dashboard                   │
│     ├── GET /health        - health check                     │
│     ├── GET /metrics       - Prometheus метрики               │
│     └── GET /api/system-info - полная информация о системе    │
│                                                               │
│  8. HTML DASHBOARD                                            │
│     └── Минимальный веб-интерфейс с real-time графиками       │
│                                                               │
╚═══════════════════════════════════════════════════════════════╝
```

🔍 Что происходит внутри:

1. При старте приложения:

```python
if __name__ == "__main__":
    app = create_app()  # Создаёт Flask приложение
    app.run(host="0.0.0.0", port=5000)
```

```
START
  ↓
create_app()
  ↓
init_db() - создаёт таблицу metrics в PostgreSQL
  ↓
app.run() - слушает порт 5000
  ↓
READY
```

---

2. При запросе GET /health:

```python
@app.route("/health")
def health():
    return jsonify({
        "status": "healthy",
        "timestamp": "2026-04-28T10:00:00Z",
        "uptime_seconds": 3600
    })
```

Поток:

```
USER → GET /health
  ↓
before_request() - засекает время начала
  ↓
health() - возвращает JSON {"status": "healthy"}
  ↓
after_request() - считает latency, инкрементирует счётчики Prometheus
  ↓
RESPONSE → {"status": "healthy", ...}
```

Время выполнения: ~5-10 мс

---

3. При запросе GET /api/system-info:

```
@app.route("/api/system-info")
def system_info():
    # 1. Проверяет Redis cache
    cached = get_cached_metrics()
    if cached:
        return jsonify(cached)

    # 2. Собирает метрики через psutil
    metrics = collect_system_metrics()

    # 3. Сохраняет в PostgreSQL
    save_metrics(...)

    # 4. Кэширует в Redis (TTL 10 сек)
    set_cached_metrics(data)

    # 5. Возвращает JSON
    return jsonify(data)
```

Поток:

```
USER → GET /api/system-info
  ↓
1. Redis cache check
   ├─ HIT → return cached data (5 ms) ✅
   └─ MISS → continue
  ↓
2. psutil.cpu_percent()      # 100 ms
   psutil.virtual_memory()   # 10 ms
   psutil.disk_usage("/")    # 50 ms
  ↓
3. Save to PostgreSQL        # 20 ms
  ↓
4. Save to Redis (TTL 10s)   # 5 ms
  ↓
5. Return JSON
  ↓
RESPONSE → {"cpu_percent": 25, "memory": {...}, ...}
```

Время выполнения:
- **С кэшем:** 5 мс ⚡
- **Без кэша:** 185 мс

---

4. При запросе GET /metrics (Prometheus):

```
@app.route("/metrics")
def metrics():
    # Обновляет gauges
    collect_system_metrics()

    # Возвращает метрики в Prometheus формате
    return Response(generate_latest(), mimetype=CONTENT_TYPE_LATEST)
```

Формат ответа:

```
# HELP app_request_total Total number of requests
# TYPE app_request_total counter
app_request_total{method="GET",endpoint="/health",http_status="200"} 1523.0

# HELP system_cpu_usage_percent Current CPU usage in percent
# TYPE system_cpu_usage_percent gauge
system_cpu_usage_percent 24.5

# HELP system_memory_usage_percent Current memory usage in percent
# TYPE system_memory_usage_percent gauge
system_memory_usage_percent 62.3
```

Prometheus скрейпит этот endpoint каждые 60 секунд.

📊 Диаграмма взаимодействия компонентов:

```
┌──────────────┐
│   Browser    │
└──────────────┘
       │
       │ GET /
       ↓
┌──────────────────────────────────────────────────────────────┐
│                       FLASK APP (port 5000)                  │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│  Routes:                                                     │
│  • GET /              → HTML Dashboard (JavaScript)         │
│  • GET /health        → {"status": "healthy"}              │
│  • GET /api/system-info → Full JSON metrics                 │
│  • GET /metrics       → Prometheus format                   │
│                                                              │
└──────────────────────────────────────────────────────────────┘
    │            │             │              │
    ↓            ↓             ↓              ↓
┌──────────┐ ┌──────────┐ ┌────────────────┐ ┌────────────────┐
│ psutil   │ │ Redis    │ │ PostgreSQL     │ │ Prometheus     │
│ (CPU,    │ │ (cache)  │ │ (persistence)  │ │ (scrape /      │
│  RAM,    │ │          │ │                │ │  metrics)      │
│  disk)   │ │          │ │                │ │                │
└──────────┘ └──────────┘ └────────────────┘ └────────────────┘
   LOCAL      Redis:      postgres:       prometheus:
  SYSTEM      6379         5432            9090
```

🎯 Что сказать на защите:

> "Я разработал Flask приложение которое собирает метрики системы через библиотеку psutil.
>
> Приложение предоставляет:
> - REST API endpoints для health check и system info
> - Prometheus метрики для мониторинга
> - HTML dashboard для визуализации
>
> Для оптимизации производительности используется Redis кэш с TTL 10 секунд - это уменьшает время ответа с 185 мс до 5 мс.
>
> Все метрики сохраняются в PostgreSQL для исторических данных.
>
> Логи пишутся в JSON формате для интеграции с Loki. Prometheus scrapes /metrics endpoint каждые 60 секунд."

📋 Таблица: Endpoints и что они делают

Endpoint	Method	Что возвращает	Кто использует	Время
`/`	GET	HTML страница с дашбордом	Пользователи в браузере	10 ms
`/health`	GET	`{"status":"healthy"}`	Docker healthcheck, K8s liveness	5 ms
`/api/system-info`	GET	JSON с CPU/RAM/Disk	Frontend (JavaScript)	5-185 ms
`/metrics`	GET	Prometheus формат метрик	Prometheus scraper	50 ms

---

2. Docker Compose

📁 Файл: `docker-compose.yml`

🎯 Что это?

Docker Compose - это инструмент для запуска multi-container приложений. Вместо того чтобы запускать каждый контейнер вручную (`docker run ...`), мы описываем все сервисы в одном YAML файле.

🏗️ Из чего состоит:

```
┌──────────────────────────────────────────────────────────┐
│              DOCKER COMPOSE STACK (7 SERVICES)              │
├──────────────────────────────────────────────────────────┤
│                                                            │
│  1. postgres       PostgreSQL database                     │
│  2. redis          Redis cache                             │
│  3. app            Flask application (твой код)            │
│  4. nginx          Reverse proxy                           │
│  5. prometheus     Metrics collector                       │
│  6. grafana        Monitoring dashboard                    │
│                                                            │
│  + 3 volumes (для persistence):                            │
│    - postgres_data                                         │
│    - prometheus_data                                       │
│    - grafana_data                                          │
│                                                            │
│  + 1 network: app-network (bridge)                         │
│                                                            │
└──────────────────────────────────────────────────────────┘
```

🔍 Разбор каждого сервиса:

1. PostgreSQL (postgres)

```yaml
postgres:
  image: postgres:15-alpine           # Образ из Docker Hub
  environment:
    POSTGRES_USER: healthuser         # Переменные окружения
    POSTGRES_PASSWORD: healthpass
    POSTGRES_DB: healthdb
  volumes:
    - postgres_data:/var/lib/postgresql/data  # Persistent storage
  networks:
    - app-network                     # Подключён к app-network
  restart: unless-stopped             # Автоперезапуск
```

Что происходит:

```
docker compose up postgres
  ↓
1. Скачивает образ postgres:15-alpine (25 MB)
  ↓
2. Создаёт volume postgres_data (если не существует)
  ↓
3. Запускает контейнер с переменными окружения
  ↓
4. PostgreSQL инициализирует БД healthdb
  ↓
5. Слушает порт 5432 (только внутри app-network)
  ↓
READY - можно подключаться по адресу postgres:5432
```

Доступ:
- **Изнутри Docker сети:** `postgres:5432`
- **С хоста:** недоступен (порты не пробрасываются)

Персистентность:
- Данные хранятся в volume `postgres_data`
- При `docker compose down` данные НЕ удаляются
- При `docker compose down -v` данные УДАЛЯЮТСЯ

---

2. Redis (redis)

```
redis:
  image: redis:7-alpine               # Лёгкий образ Redis
  networks:
    - app-network
  restart: unless-stopped
```

Что происходит:

```
docker compose up redis
  ↓
1. Скачивает образ redis:7-alpine (10 MB)
  ↓
2. Запускает Redis server
  ↓
3. Слушает порт 6379 (внутри app-network)
  ↓
READY - кэш доступен по адресу redis:6379
```

Использование:
- Flask app подключается: `redis://redis:6379/0`
- Хранит метрики с TTL 10 секунд
- **БЕЗ персистентности** - данные в RAM, пропадают при перезапуске

---

3. Flask App (app)

```yaml
app:
  build: .                            # Собирает из локального Dockerfile
  image: health-dashboard-app
  environment:
    DATABASE_URL: postgresql://...
    REDIS_URL: redis://redis:6379/0
    FLASK_ENV: production
  depends_on:                         # Зависимости
    - postgres
    - redis
  networks:
    - app-network
  mem_limit: 256m                     # Лимит памяти
  healthcheck:                        # Проверка здоровья
    test: ["CMD", "python3", "-c", "..."]
    interval: 30s
```

Что происходит:

```
docker compose up app
  ↓
1. Проверяет зависимости: postgres, redis (должны быть запущены)
  ↓
2. Собирает образ из Dockerfile (если не существует)
  ↓
3. Создаёт контейнер с переменными окружения
  ↓
4. Запускает: gunicorn app.wsgi:app -b 0.0.0.0:5000
  ↓
5. Каждые 30 секунд запускает healthcheck:
   curl http://localhost:5000/health
  ↓
READY - Flask app слушает порт 5000
```

Healthcheck:

```
# Каждые 30 секунд Docker выполняет:
python3 -c "import urllib.request; urllib.request.urlopen('http://localhost:5000/
health')"

# Если успешно → контейнер healthy ✅
# Если 3 раза подряд fail → контейнер unhealthy ❌ (Docker может перезапустить)
```

---

4. Nginx (nginx)

```yaml
nginx:
  image: nginx:alpine
  ports:
    - "80:80"                        # Пробрасывает порт 80 наружу
  volumes:
    - ./nginx/nginx.conf:/etc/nginx/nginx.conf:ro  # Читает конфиг с хоста
  depends_on:
    - app
  networks:
    - app-network
  mem_limit: 64m
```

Что происходит:

```
docker compose up nginx
  ↓
1. Скачивает nginx:alpine (5 MB)
  ↓
2. Монтирует конфиг ./nginx/nginx.conf внутрь контейнера
  ↓
3. Запускает Nginx с этим конфигом
  ↓
4. Открывает порт 80 наружу (host:80 → container:80)
  ↓
READY - все запросы на http://localhost идут в Nginx
```

Роль Nginx:

```
USER → http://54.93.95.178/health
  ↓
Nginx (порт 80)
  ↓
proxy_pass http://app:5000
  ↓
Flask App (порт 5000)
  ↓
RESPONSE ← {"status": "healthy"}
```

Зачем нужен Nginx?

1. **Reverse Proxy:** Скрывает Flask app за Nginx
2. **Security headers:** Добавляет X-Frame-Options, X-XSS-Protection
3. **Load balancing:** Может распределять запросы на несколько app инстансов
4. **SSL/TLS:** Можно добавить HTTPS сертификат (Let’s Encrypt)

---

5. Prometheus (prometheus)

```yaml
prometheus:
  image: prom/prometheus:latest
  ports:
    - "9090:9090"                     # Доступен снаружи
  volumes:
    - ./monitoring/prometheus.yml:/etc/prometheus/prometheus.yml:ro
    - prometheus_data:/prometheus     # Persistent storage
  command:
    - '--config.file=/etc/prometheus/prometheus.yml'
    - '--storage.tsdb.path=/prometheus'
    - '--storage.tsdb.retention.time=3h'  # Хранит данные 3 часа
  networks:
    - app-network
  mem_limit: 128m
```

Что происходит:

```
docker compose up prometheus
  ↓
1. Скачивает prom/prometheus:latest (50 MB)
  ↓
2. Читает конфиг ./monitoring/prometheus.yml
  ↓
3. Каждые 60 секунд scrapes targets:
   - http://localhost:9090/metrics (self)
   - http://app:5000/metrics (Flask)
  ↓
4. Сохраняет метрики в /prometheus (TSDB)
  ↓
5. Предоставляет UI на порту 9090
  ↓
READY - можно смотреть графики на http://localhost:9090
```

Конфигурация (prometheus.yml):

```yaml
scrape_configs:
  - job_name: 'prometheus'
    static_configs:
      - targets: ['localhost:9090']   # Мониторит сам себя

  - job_name: 'flask-app'
    static_configs:
      - targets: ['app:5000']         # Мониторит Flask
```

Цикл scraping:

```
Время     | Действие
---------|-------------------------------------------------
00:00:00 | Prometheus запускается
00:00:00 | Читает prometheus.yml
00:00:00 | Scrape app:5000/metrics → сохраняет метрики
00:01:00 | Scrape app:5000/metrics → сохраняет метрики
00:02:00 | Scrape app:5000/metrics → сохраняет метрики
...      | (каждые 60 секунд)
03:00:00 | Удаляет метрики старше 3 часов (retention)
```

---

6. Grafana (grafana)

```yaml
grafana:
  image: grafana/grafana:10.4.7
  ports:
    - "3000:3000"
  environment:
    - GF_SECURITY_ADMIN_USER=admin
    - GF_SECURITY_ADMIN_PASSWORD=admin
    - GF_USERS_ALLOW_SIGN_UP=false
  volumes:
    - grafana_data:/var/lib/grafana
    - ./monitoring/grafana/provisioning:/etc/grafana/provisioning:ro
    - ./monitoring/grafana/dashboards:/var/lib/grafana/dashboards:ro
  depends_on:
    - prometheus
  networks:
    - app-network
  mem_limit: 128m
```

Что происходит:

```
docker compose up grafana
  ↓
1. Скачивает grafana/grafana:10.4.7 (80 MB)
  ↓
2. Создаёт volume grafana_data
  ↓
3. Читает provisioning конфиги:
   - datasources.yaml (подключает Prometheus)
   - dashboards (импортирует готовые дашборды)
  ↓
4. Запускается с admin:admin
  ↓
5. Открывает UI на порту 3000
  ↓
READY - можно смотреть дашборды на http://localhost:3000
```

Логин:
- **Username:** `admin`
- **Password:** `admin`

🔄 Как работает весь стек:

```
┌──────────────────────────────────────────────────────────────────────┐
│                        DOCKER COMPOSE STACK                          │
├──────────────────────────────────────────────────────────────────────┤
│                                                                      │
│   ┌──────────────────────────────────────────────────────────┐       │
│   │  USER REQUEST                                            │       │
│   │  http://54.93.95.178/health                              │       │
│   └──────────────────────────────────────────────────────────┘       │
│                      │                                               │
│                      ↓                                               │
│   ┌──────────────────────────────────────────────────────────┐       │
│   │  NGINX (port 80)                                         │       │
│   │  - Reverse proxy                                         │       │
│   │  - Security headers                                      │       │
│   │  - proxy_pass → app:5000                                 │       │
│   └──────────────────────────────────────────────────────────┘       │
│                      │                                               │
│                      ↓                                               │
│   ┌──────────────────────────────────────────────────────────┐       │
│   │  FLASK APP (port 5000)                                   │       │
│   │  - Обрабатывает запрос                                   │       │
│   │  - Проверяет Redis cache                                 │       │
│   │  - Собирает метрики (psutil)                             │       │
│   │  - Сохраняет в PostgreSQL                                │       │
│   └──────────────────────────────────────────────────────────┘       │
│       │               │               │                              │
│       ↓               ↓               ↓                              │
│   ┌─────────┐     ┌──────────┐    ┌──────────┐                      │
│   │ Redis   │     │Postgres  │    │Prometheus│                      │
│   │ :6379   │     │  :5432   │    │  :9090   │                      │
│   └─────────┘     └──────────┘    └──────────┘                      │
│    (cache)       (database)           │                              │
│                                       ↓                              │
│                                   ┌──────────┐                       │
│                                   │ Grafana  │                       │
│                                   │  :3000   │                       │
│                                   └──────────┘                       │
│                                  (visualize)                         │
│                                                                      │
└──────────────────────────────────────────────────────────────────────┘
      ↑                                                       ↑
      │                                                       │
   app-network (Docker bridge)                          Volumes:
   All containers can talk                            - postgres_data
   to each other by name                             - prometheus_data
                                                      - grafana_data
```

📋 Команды Docker Compose:

```
# Запустить все сервисы
docker compose up -d

# Что происходит:
# 1. Создаёт network app-network
# 2. Создаёт volumes (если не существуют)
# 3. Запускает контейнеры в правильном порядке:
#    postgres → redis → app → nginx → prometheus → grafana
# 4. Флаг -d (detached) = фоновый режим

# Посмотреть статус
docker compose ps

# Пример вывода:
# NAME                   IMAGE                     STATUS         PORTS
# health-dashboard-app   health-dashboard-app      Up 2 minutes   5000/tcp
# nginx                  nginx:alpine              Up 2 minutes   0.0.0.0:80->80/tcp
# postgres               postgres:15-alpine        Up 2 minutes   5432/tcp
# redis                  redis:7-alpine            Up 2 minutes   6379/tcp
# prometheus             prom/prometheus:latest    Up 2 minutes   0.0.0.0:9090->9090/tcp
# grafana                grafana/grafana:10.4.7    Up 2 minutes   0.0.0.0:3000->3000/tcp

# Посмотреть логи
docker compose logs app -f

# Остановить все сервисы
docker compose down

# Остановить и УДАЛИТЬ volumes (все данные пропадут!)
docker compose down -v

# Пересобрать образы
docker compose build

# Перезапустить один сервис
docker compose restart app
```

🎯 Что сказать на защите:

> “Я использовал Docker Compose для оркестрации multi-container приложения.
>
> Стек состоит из 7 сервисов:
> - PostgreSQL - база данных
> - Redis - кэш для оптимизации
> - Flask App - моё приложение
> - Nginx - reverse proxy и security
> - Prometheus - сбор метрик
> - Grafana - визуализация
>
> Все сервисы подключены к единой Docker bridge сети app-network и могут общаться по именам (например, `app:5000`, `postgres:5432`).
>
> Для персистентности данных использую 3 volumes:
> - postgres_data - хранит БД
> - prometheus_data - хранит метрики
> - grafana_data - хранит дашборды
>
> При `docker compose down` данные сохраняются, при `docker compose down -v` удаляются.
>
> Запуск всего стека - одна команда: `docker compose up -d`. Это упрощает деплой и делает инфраструктуру reproducible.”

📊 Таблица: Сервисы и их роль

Сервис	Образ	Порты	Роль	RAM лимит
postgres	postgres:15-alpine	5432 (внутри)	База данных	-
redis	redis:7-alpine	6379 (внутри)	Кэш	-
app	health-dash-board-app	5000 (внутри)	Flask приложение	256 MB
nginx	nginx:alpine	80 (наружу)	Reverse proxy	64 MB
prometheus	prom/prometh-eus:latest	9090 (наружу)	Метрики	128 MB
grafana	grafana/grafana:10.4.7	3000 (наружу)	Визуализация	128 MB

Total RAM: ~576 MB (для всего стека)

---

3. CI/CD (GitHub Actions)

📁 Файл: `.github/workflows/ci-cd.yml`

🎯 Что это?

CI/CD Pipeline - это автоматизированный процесс который:
- **CI (Continuous Integration):** Проверяет код (тесты, линтинг) при каждом push
- **CD (Continuous Deployment):** Автоматически деплоит на сервер если тесты прошли

🏗️ Структура pipeline:

```
┌─────────────────────────────────────────────────────────────┐
│              CI/CD PIPELINE (3 STAGES)                      │
├─────────────────────────────────────────────────────────────┤
│                                                             │
│  TRIGGER: push/pull_request → main branch                   │
│                                                             │
│  ┌──────────────────────────────────────────────────────┐   │
│  │  STAGE 1: TEST 🧪                                    │   │
│  │  ────────────────────────────────────────────────    │   │
│  │  - Checkout code                                     │   │
│  │  - Setup Python 3.11                                 │   │
│  │  - Install dependencies (pip install -r requirements) │   │
│  │  - Run pytest (unit tests)                           │   │
│  │  - Generate coverage report                          │   │
│  │                                                      │   │
│  │  If FAIL → Pipeline STOPS ❌                         │   │
│  │  If PASS → Continue to BUILD ✅                      │   │
│  └──────────────────────────────────────────────────────┘   │
│                          ↓                                  │
│  ┌──────────────────────────────────────────────────────┐   │
│  │  STAGE 2: BUILD 🐳                                   │   │
│  │  ────────────────────────────────────────────────    │   │
│  │  - Checkout code                                     │   │
│  │  - Login to Docker Hub                               │   │
│  │  - Build Docker image from Dockerfile                │   │
│  │  - Tag: latest + git commit SHA                      │   │
│  │  - Push to Docker Hub                                │   │
│  │                                                      │   │
│  │  Only runs: if branch = main AND tests passed       │   │
│  └──────────────────────────────────────────────────────┘   │
│                          ↓                                  │
│  ┌──────────────────────────────────────────────────────┐   │
│  │  STAGE 3: DEPLOY 🚀                                  │   │
│  │  ────────────────────────────────────────────────    │   │
│  │  - SSH to AWS server (via secrets)                   │   │
│  │  - cd /opt/health-dashboard                          │   │
│  │  - git pull origin main                              │   │
│  │  - docker compose down                               │   │
│  │  - docker compose up -d --build                      │   │
│  │  - Wait 30 seconds                                   │   │
│  │  - Health check (curl http://localhost/health)       │   │
│  │                                                      │   │
│  │  Only runs: if branch = main AND build successful   │   │
│  └──────────────────────────────────────────────────────┘   │
│                                                             │
└─────────────────────────────────────────────────────────────┘
```

🔍 Разбор каждого этапа:

STAGE 1: Test 🧪

```yaml
test:
  name: 🧪 Run Tests
  runs-on: ubuntu-latest    # GitHub предоставляет VM с Ubuntu

  steps:
    - name: 📥 Checkout code
      uses: actions/checkout@v4

    - name: 🐍 Set up Python
      uses: actions/setup-python@v5
      with:
        python-version: '3.11'

    - name: 📦 Install dependencies
      run: |
        pip install -r requirements.txt
        pip install pytest pytest-cov

    - name: ✅ Run tests
      run: |
        pytest tests/ -v --cov=app --cov-report=term-missing
```

Что происходит:

```
GitHub Action Runner (Ubuntu VM)
  ↓
1. Клонирует репозиторий (actions/checkout@v4)
  ↓
2. Устанавливает Python 3.11
  ↓
3. pip install -r requirements.txt
   - Flask, gunicorn, psutil, psycopg2, redis, ...
  ↓
4. pip install pytest pytest-cov
  ↓
5. Запускает тесты:
   pytest tests/ -v --cov=app
  ↓
   Running: test_health_endpoint_returns_200 ... PASSED ✅
   Running: test_health_response_structure ... PASSED ✅
   Running: test_system_info ... PASSED ✅
   ...
   12 tests total
  ↓
6. Генерирует coverage report:
   app/app.py ...................... 85%
   TOTAL ............................ 85%
  ↓
✅ All tests passed → Continue to BUILD
❌ Any test failed → STOP pipeline, notify developer
```

Время выполнения: ~2-3 минуты

STAGE 2: Build 🐳

```yaml
build:
  name: 🐳 Build and Push Docker Image
  runs-on: ubuntu-latest
  needs: test                 # Запускается только если test прошёл
  if: github.ref == 'refs/heads/main'  # Только для main ветки

  steps:
    - name: 🔐 Login to Docker Hub
      uses: docker/login-action@v3
      with:
        username: ${{ secrets.DOCKER_USERNAME }}
        password: ${{ secrets.DOCKER_PASSWORD }}

    - name: 🏗️ Build and push
      uses: docker/build-push-action@v5
      with:
        context: .
        push: true
        tags: |
          oskalibriya/health-dashboard:latest
          oskalibriya/health-dashboard:${{ github.sha }}
```

Что происходит:

```
GitHub Action Runner
  ↓
1. Проверяет: test job прошёл? Да ✅
  ↓
2. Проверяет: ветка = main? Да ✅
  ↓
3. Login to Docker Hub:
   docker login -u oskalibriya -p ****
  ↓
4. Build Docker image:
   docker build -t oskalibriya/health-dashboard:latest .

   Dockerfile выполняется:
   [Stage 1: Builder]
   FROM python:3.11-slim
   RUN apt-get install gcc python3-dev
   COPY requirements.txt .
   RUN pip install --prefix=/install -r requirements.txt

   [Stage 2: Production]
   FROM python:3.11-slim
   COPY --from=builder /install /usr/local
   COPY app/ ./app/

   Image size: ~150 MB
  ↓
5. Tag с commit SHA:
   docker tag oskalibriya/health-dashboard:latest \
               oskalibriya/health-dashboard:abc123def
  ↓
6. Push to Docker Hub:
   docker push oskalibriya/health-dashboard:latest
   docker push oskalibriya/health-dashboard:abc123def
  ↓
✅ Image available on Docker Hub
```

Время выполнения: ~3-5 минут

Зачем 2 тега?

- **latest:** Всегда указывает на последнюю версию (удобно для деплоя)
- **git SHA:** Специфическая версия (можно откатиться: `docker pull oskalibriya/health-dashboard:abc123def`)

STAGE 3: Deploy 🚀

```yaml
deploy:
  name: 🚀 Deploy to Server
  runs-on: ubuntu-latest
  needs: build                # Запускается только если build прошёл
  if: github.ref == 'refs/heads/main'

  steps:
    - name: 🚀 Deploy to Server
      uses: appleboy/ssh-action@v1
      with:
        host: ${{ secrets.SERVER_HOST }}        # 35.158.171.183
        username: ${{ secrets.SERVER_USER }}    # ec2-user
        key: ${{ secrets.SSH_PRIVATE_KEY }}     # my-devops-key.pem
        script: |
          cd /opt/health-dashboard

          echo "📥 Pulling latest changes..."
          git pull origin main

          if [ ! -f .env ]; then
            echo "📝 Creating .env file..."
            cp .env.example .env
          fi

          echo "🛑 Stopping old containers..."
          docker compose down

          echo "🚀 Starting new containers..."
          docker compose up -d --build

          echo "⏳ Waiting 30 seconds for services to fully start..."
          sleep 30

          echo "🏥 Application Health Check:"
          if curl -sf http://localhost/health > /dev/null 2>&1; then
            echo "✅ Application is HEALTHY"
            curl -s http://localhost/health | jq '.'
          else
            echo "❌ Application health check FAILED"
            docker compose logs app --tail=20
            exit 1
          fi

          echo "✅ Deployment successful!"
```

Что происходит:

```
GitHub Action Runner
  ↓
1. Проверяет: build job прошёл? Да ✅
  ↓
2. SSH подключение к AWS сервер:
   ssh ec2-user@35.158.171.183 -i my-devops-key.pem
  ↓
3. На сервере выполняет команды:

  cd /opt/health-dashboard
  ↓
  git pull origin main
  (скачивает последний код с GitHub)
  ↓
  docker compose down
  (останавливает старые контейнеры)
  ↓
  docker compose up -d --build
  (запускает новые контейнеры)
  ↓
  sleep 30
  (ждёт пока всё стартует)
  ↓
  curl http://localhost/health
  (проверяет что приложение работает)
  ↓
  ✅ {"status": "healthy"} → Success
  ❌ Connection refused → Fail, rollback
```

Время выполнения: ~2-3 минуты

🔄 Полный цикл от commit до deploy:

```
DEVELOPER (Local Machine)
  ↓
1. Пишет код, делает commit
   $ git add .
   $ git commit -m "Add new feature"
   $ git push origin main
  ↓
2. GitHub получает push
  ↓
──────────────────────────────────────────────────────────
GITHUB ACTIONS (Cloud)
──────────────────────────────────────────────────────────
  ↓
3. STAGE 1: TEST (2-3 min)
   ✅ All 12 tests passed
  ↓
4. STAGE 2: BUILD (3-5 min)
   ✅ Docker image built and pushed to Docker Hub
  ↓
5. STAGE 3: DEPLOY (2-3 min)
   SSH → AWS Server
   ✅ Application deployed and healthy
  ↓
──────────────────────────────────────────────────────────
AWS SERVER (35.158.171.183)
──────────────────────────────────────────────────────────
  ↓
6. Old containers stopped
  ↓
7. New containers started
  ↓
8. Application running on http://35.158.171.183
  ↓
──────────────────────────────────────────────────────────
TOTAL TIME: 7-11 минут (автоматически!)
──────────────────────────────────────────────────────────
```

📊 GitHub Secrets (переменные окружения):

```
GitHub Repository → Settings → Secrets and variables → Actions
```

Secret Name	Value	Для чего
DOCKER_USERNAME	oskalibriya	Login в Docker Hub
DOCKER_PASSWORD	4da7CB1234/	Password для Docker Hub
SERVER_HOST	35.158.171.183	IP адрес AWS сервера
SERVER_USER	ec2-user	SSH username
SSH_PRIVATE_KEY	`-----BEGIN RSA...`	SSH ключ для подключения

Эти secrets НЕ видны в логах и коде - они зашифрованы GitHub.

🎯 Что сказать на защите:

> "Я настроил полностью автоматизированный CI/CD pipeline через GitHub Actions.
>
> Pipeline состоит из 3 этапов:**
>
> **1. TEST (2-3 мин):**
> - Запускает 12 unit tests
> - Проверяет code coverage (85%)
> - Если хотя бы один тест fail - pipeline останавливается
>
> **2. BUILD (3-5 мин):**
> - Собирает Docker образ из Dockerfile
> - Использует multi-stage build для оптимизации
> - Загружает образ на Docker Hub с двумя тегами: `latest` и `git SHA`
>
> **3. DEPLOY (2-3 мин):**
> - Подключается по SSH к AWS серверу
> - Выполняет `git pull` для получения последнего кода
> - Перезапускает контейнеры через `docker compose up -d`
> - Делает health check для проверки что всё работает
>
> Весь процесс от commit до production - 7-11 минут автоматически.
>
> При любой ошибке на любом этапе - pipeline останавливается и отправляет уведомление.
>
> Это реализует Continuous Integration и Continuous Deployment - ключевые практики DevOps."

📋 Таблица: Этапы и время

Этап	Что делает	Время	Если fail
Test	Запускает pytest, проверяет код	2-3 min	Pipeline stops ❌
Build	Собирает Docker образ	3-5 min	Pipeline stops ❌
Deploy	Деплоит на AWS	2-3 min	Rollback (старые контейнеры не удалялись)
Total	Весь цикл	**7-11 min**	❌

---

4. Terraform (Infrastructure as Code)

📁 Файл: `terraform/main.tf`

🎯 Что это?

Terraform - это инструмент для описания инфраструктуры в виде кода (Infrastructure as Code). Вместо того чтобы создавать EC2 инстанс вручную через AWS Console, мы описываем его в `.tf` файле и Terraform автоматически создаёт/обновляет/удаляет ресурсы.

🏗️ Что создаёт Terraform:

```
╔═══════════════════════════════════════════════════════════╗
║                   AWS INFRASTRUCTURE                      ╒
║                   (eu-central-1 region)                   ╒
╟═══════════════════════════════════════════════════════════╢
╒                                                           ╒
╒  1. TLS Private Key                                      ╒
╒     ╙ RSA 4096-bit SSH key pair                           ╒
╒                                                           ╒
╒  2. AWS Key Pair                                         ╒
╒     ╙ my-devops-key (для SSH доступа)                     ╒
╒                                                           ╒
╒  3. Security Group                                       ╒
╒     ╟ Ingress: 22 (SSH)                                   ╒
╒     ╟ Ingress: 80 (HTTP)                                  ╒
╒     ╟ Ingress: 443 (HTTPS)                                ╒
╒     ╟ Ingress: 3000 (Grafana)                             ╒
╒     ╟ Ingress: 5000 (Flask App)                           ╒
╒     ╟ Ingress: 9090 (Prometheus)                          ╒
╒     ╙ Egress: * (all outbound)                            ╒
╒                                                           ╒
╒  4. EC2 Instance                                        ╒
╒     ╟ Type: t3.micro (2 vCPU, 1 GB RAM)                   ╒
╒     ╟ AMI: Amazon Linux 2023                              ╒
╒     ╟ Storage: 30 GB gp3 SSD                              ╒
╒     ╟ User Data: Install Docker, Git, Docker Compose      ╒
╒     ╙ Tags: Name=health-dashboard-server                  ╒
╒                                                           ╒
╒  5. Elastic IP                                           ╒
╒     ╙ Static public IP (не меняется при перезапуске)      ╒
╒                                                           ╒
╚═══════════════════════════════════════════════════════════╝
```

🔍 Разбор каждого ресурса:

1. TLS Private Key (SSH ключ)

```
resource "tls_private_key" "ssh_key" {
  algorithm = "RSA"
  rsa_bits  = 4096
}
```

Что происходит:

```
terraform apply
  ↓
Terraform генерирует RSA ключ 4096-bit
  ↓
Приватный ключ: -----BEGIN RSA PRIVATE KEY-----
                MIIJKQIBAAKCAgEAwL8M9...
                -----END RSA PRIVATE KEY-----
                (сохраняется в terraform.tfstate)
  ↓
Публичный ключ:  ssh-rsa AAAAB3NzaC1yc2EAAAADAQA...
                 (будет загружен в AWS Key Pair)
```

Зачем нужен?

- Для SSH подключения к EC2 серверу
- GitHub Actions использует приватный ключ для деплоя

---

2. AWS Key Pair

```
resource "aws_key_pair" "deployer" {
  key_name   = "my-devops-key"
  public_key = tls_private_key.ssh_key.public_key_openssh
}
```

Что происходит:

```
terraform apply
  ↓
Terraform загружает публичный ключ в AWS
  ↓
AWS создаёт Key Pair "my-devops-key"
  ↓
Этот ключ можно прикрепить к EC2 instance
  ↓
Подключение: ssh -i my-devops-key.pem ec2-user@IP
```

---

3. Security Group (Firewall)

```
resource "aws_security_group" "health_dashboard_sg" {
  name = "health-dashboard-sg"

  # SSH
  ingress {
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  # HTTP
  ingress {
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  # ... (ещё 4 правила для 443, 3000, 5000, 9090)

  # Egress: allow all outbound
  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}
```

Что происходит:

```
terraform apply
  ↓
AWS создаёт Security Group "health-dashboard-sg"
  ↓
Правила Ingress (входящие):
  22   (SSH)         ← 0.0.0.0/0 (весь интернет) ⚠️
  80   (HTTP)        ← 0.0.0.0/0
  443  (HTTPS)       ← 0.0.0.0/0
  3000 (Grafana)     ← 0.0.0.0/0
  5000 (Flask)       ← 0.0.0.0/0
  9090 (Prometheus)  ← 0.0.0.0/0
  ↓
Правила Egress (исходящие):
  *    (all)         → 0.0.0.0/0 (весь интернет) ✅
```

Security Group = виртуальный firewall вокруг EC2

```
ИНТЕРНЕТ
  ↓
  Port 22 (SSH) ✅ allowed
  Port 80 (HTTP) ✅ allowed
  Port 3000 (Grafana) ✅ allowed
  Port 8888 (random) ❌ blocked
  ↓
┌──────────────────────┐
│   EC2 INSTANCE       │
│   Security Group     │
│   (firewall)         │
└──────────────────────┘
  ↓
  Outbound: all ✅
```

---

4. EC2 Instance

```
resource "aws_instance" "health_dashboard" {
  ami                    = data.aws_ami.amazon_linux.id  # Amazon Linux 2023
  instance_type          = "t3.micro"                    # 2 vCPU, 1 GB RAM
  vpc_security_group_ids = [aws_security_group.health_dashboard_sg.id]
  key_name               = aws_key_pair.deployer.key_name

  user_data = <<-EOF
    #!/bin/bash
    set -e
    yum update -y
    yum install -y docker git
    systemctl start docker
    systemctl enable docker

    # Install Docker Compose
    curl -SL "https://github.com/docker/compose/releases/latest/download/docker-
compose-$(uname -s)-$(uname -m)" -o /usr/local/bin/docker-compose
    chmod +x /usr/local/bin/docker-compose

    usermod -aG docker ec2-user
  EOF

  root_block_device {
    volume_size = 30      # 30 GB
    volume_type = "gp3"   # SSD
  }
}
```

Что происходит:

```
terraform apply
  ↓
AWS ищет последний AMI Amazon Linux 2023
  ↓
AWS создаёт EC2 instance:
  - Type: t3.micro ($0.0104/hour = ~$7.5/month)
  - CPU: 2 vCPU (Intel Xeon)
  - RAM: 1 GB
  - Disk: 30 GB gp3 SSD
  - Region: eu-central-1 (Frankfurt)
  - AZ: eu-central-1a (случайно)
  ↓
AWS присоединяет Security Group
  ↓
AWS присоединяет SSH Key Pair
  ↓
AWS запускает user_data script:

  1. yum update -y           (обновляет пакеты, 2 min)
  2. yum install docker git  (устанавливает Docker и Git, 1 min)
  3. systemctl start docker  (запускает Docker daemon)
  4. curl ... docker-compose (скачивает Docker Compose, 30 sec)
  5. usermod -aG docker ...  (добавляет ec2-user в docker группу)
  ↓
EC2 instance запущен и готов к работе! (Total: ~5-7 min)
  ↓
Публичный IP: 35.158.171.183 (случайный, меняется при restart)
```

User Data Script = команды которые выполняются ОДИН РАЗ при первом запуске EC2

---

5. Elastic IP (Static IP)

```
resource "aws_eip" "app_eip" {
  domain = "vpc"
}

resource "aws_eip_association" "app_eip_assoc" {
  instance_id   = aws_instance.health_dashboard.id
  allocation_id = aws_eip.app_eip.id
}
```

Что происходит:

```
terraform apply
  ↓
AWS создаёт Elastic IP (статический IP адрес)
  ↓
Elastic IP: 35.158.171.183
  ↓
AWS присоединяет Elastic IP к EC2 instance
  ↓
Теперь IP не меняется при:
  - EC2 restart
  - EC2 stop/start
  - Только при terraform destroy
```

Зачем нужен Elastic IP?

```
БЕЗ Elastic IP:
EC2 start   → IP: 3.127.155.114
EC2 restart → IP: 18.195.72.89  (ИЗМЕНИЛСЯ! ❌)
              GitHub Secrets нужно обновлять вручную
              DNS записи нужно менять

С Elastic IP:
EC2 start   → IP: 35.158.171.183
EC2 restart → IP: 35.158.171.183  (НЕ ИЗМЕНИЛСЯ ✅)
              Ничего не нужно менять
```

🔄 Terraform Workflow:

```
# 1. Инициализация (первый раз)
$ terraform init

# Что происходит:
# - Скачивает провайдеры (AWS, TLS)
# - Создаёт .terraform/ папку
# - Готовит backend (где хранить state)

# 2. План (preview изменений)
$ terraform plan

# Выход:
Terraform will perform the following actions:

  # aws_instance.health_dashboard will be created
  + resource "aws_instance" "health_dashboard" {
      + ami                          = "ami-0c55b159cbfafe1f0"
      + instance_type                = "t3.micro"
      ...
    }

  # aws_security_group.health_dashboard_sg will be created
  + resource "aws_security_group" "health_dashboard_sg" {
      + name = "health-dashboard-sg"
      ...
    }

Plan: 5 to add, 0 to change, 0 to destroy.

# 3. Применить изменения
$ terraform apply

# Terraform спросит подтверждение:
Do you want to perform these actions? (yes/no)
> yes

# Выполняется:
aws_security_group.health_dashboard_sg: Creating...
aws_security_group.health_dashboard_sg: Creation complete after 3s [id=sg-0abcd1234]
aws_key_pair.deployer: Creating...
aws_key_pair.deployer: Creation complete after 1s [id=my-devops-key]
aws_instance.health_dashboard: Creating...
aws_instance.health_dashboard: Still creating... [10s elapsed]
aws_instance.health_dashboard: Still creating... [20s elapsed]
...
aws_instance.health_dashboard: Creation complete after 45s [id=i-003f45cb781d8f182]
aws_eip.app_eip: Creating...
aws_eip.app_eip: Creation complete after 2s [id=eipalloc-0abc123]
aws_eip_association.app_eip_assoc: Creating...
aws_eip_association.app_eip_assoc: Creation complete after 1s

Apply complete! Resources: 5 added, 0 changed, 0 destroyed.

Outputs:

instance_id = "i-003f45cb781d8f182"
public_ip   = "35.158.171.183"
ssh_command = "ssh -i my-devops-key.pem ec2-user@35.158.171.183"

# 4. Посмотреть state
$ terraform show
```

```
# 5. Удалить всю инфраструктуру
$ terraform destroy

# Terraform спросит подтверждение:
Do you really want to destroy all resources? (yes/no)
> yes

# Удалит ВСЁ:
aws_eip_association.app_eip_assoc: Destroying... [id=eipassoc-0xyz]
aws_instance.health_dashboard: Destroying... [id=i-003f45cb781d8f182]
...
Destroy complete! Resources: 5 destroyed.
```

📊 Terraform State:

```
╔═══════════════════════════════════════════════════════════╗
║                      TERRAFORM STATE                      ╒
╟═══════════════════════════════════════════════════════════╢
│                                                           │
│  terraform.tfstate (JSON файл)                            │
│  ┌─────────────────────────────────────────────────────┐  │
│  {                                                        │
│    "version": 4,                                          │
│    "resources": [                                         │
│      {                                                    │
│        "type": "aws_instance",                            │
│        "name": "health_dashboard",                        │
│        "instances": [{                                    │
│          "attributes": {                                  │
│            "id": "i-003f45cb781d8f182",                   │
│            "public_ip": "35.158.171.183",                 │
│            "instance_type": "t3.micro",                   │
│            ...                                            │
│          }                                                │
│        }]                                                 │
│      },                                                   │
│      ...                                                  │
│    ]                                                      │
│  }                                                        │
│                                                           │
│  ✅ State = single source of truth                        │
│  ✅ Terraform знает что создано, может обновлять/удалять  │
│  ❌ НЕ КОММИТИТЬ в Git! (содержит приватные ключи)        │
│                                                           │
╘═══════════════════════════════════════════════════════════╛
```

🎯 Что сказать на защите:

> "Я использовал Terraform для описания всей AWS инфраструктуры в виде кода (Infrastructure as Code).
>
> Terraform создаёт:
> - EC2 instance (t3.micro, Amazon Linux 2023, 30 GB SSD)
> - Security Group с правилами для портов 22, 80, 443, 3000, 5000, 9090
> - SSH Key Pair (RSA 4096-bit)
> - Elastic IP (статический IP адрес)
>
> User Data script автоматически устанавливает Docker, Git, и Docker Compose при первом запуске.
>
> Вся инфраструктура создаётся одной командой: `terraform apply`. Если что-то сломается, могу пересоздать за 5 минут: `terraform destroy && terraform apply`.
>
> Terraform хранит состояние в `terraform.tfstate` файле - это позволяет обновлять ресурсы инкрементально (например, изменить тип instance с t3.micro на t3.small).
>
> Это Infrastructure as Code - могу версионировать инфраструктуру в Git, делать code review, и reproducible deploys."

📋 Таблица: Созданные ресурсы

Ресурс	Type	ID	Стоимость
EC2 Instance	t3.micro	i-003f45cb781d8f182	$0.0104/hour (~$7.5/month)
Elastic IP	Static IP	35.158.171.183	$0 (пока attached)
Security Group	Firewall	sg-0abcd1234	FREE
Key Pair	SSH key	my-devops-key	FREE
EBS Volume	30 GB gp3	vol-0xyz	$2.4/month
TOTAL			**~$10/month**

---

5. Ansible (Configuration Management)

📁 Файлы: `ansible/playbook.yml`, `ansible/roles/app/tasks/main.yml`

🎯 Что это?

Ansible - это инструмент для автоматизации настройки серверов. Вместо того чтобы вручную SSH на сервер и выполнять команды, Ansible делает это автоматически по "плейбуку" (playbook).

🏗️ Структура Ansible:

```
ansible/
├── playbook.yml              # Главный playbook
├── inventory                 # Список серверов
└── roles/
    ├── docker/               # Роль: установка Docker
    │   └── tasks/
    │       └── main.yml
    └── app/                  # Роль: деплой приложения
        └── tasks/
            └── main.yml
```

🔍 Что делает Ansible:

```
╔═══════════════════════════════════════════════════════════╗
║                      ANSIBLE PLAYBOOK                     ║
╟═══════════════════════════════════════════════════════════╢
║                                                           ║
║  TARGET: webservers (AWS EC2)                             ║
║                                                           ║
║  TASKS:                                                   ║
║  ╔═══════════════════════════════════════════════════╗    ║
║  ║  1. Update system packages (yum update)            ║    ║
║  ║  2. Install Git                                    ║    ║
║  ║  3. Install Firewalld                              ║    ║
║  ║  4. Open ports: 22, 80, 443, 3000, 5000, 9090      ║    ║
║  ╚═══════════════════════════════════════════════════╝    ║
║                                                           ║
║  ROLES:                                                   ║
║  ╔═══════════════════════════════════════════════════╗    ║
║  ║  ROLE: docker                                       ║    ║
║  ║  ╟─ Install Docker                                  ║    ║
║  ║  ╟─ Install Docker Compose                          ║    ║
║  ║  ╙─ Add ec2-user to docker group                    ║    ║
║  ╚═══════════════════════════════════════════════════╝    ║
║  ╔═══════════════════════════════════════════════════╗    ║
║  ║  ROLE: app                                          ║    ║
║  ║  ╟─ Create /opt/health-dashboard directory          ║    ║
║  ║  ╟─ Clone Git repository                             ║    ║
║  ║  ╟─ Copy .env file                                  ║    ║
║  ║  ╟─ Run: docker compose up -d --build               ║    ║
║  ║  ╙─ Wait for /health endpoint to be 200             ║    ║
║  ╚═══════════════════════════════════════════════════╝    ║
║                                                           ║
╚═══════════════════════════════════════════════════════════╝
```

📋 Разбор playbook.yml:

```yaml
- name: Deploy Health Dashboard
  hosts: webservers          # Какие серверы (из inventory)
  become: yes                # Запускать с sudo

  vars:
    app_dir: /opt/health-dashboard
    docker_compose_version: "2.24.0"

  roles:
    - docker                 # Вызывает роль docker
    - app                    # Вызывает роль app

  tasks:
    - name: Update system packages
      yum:
        name: "*"
        state: latest
      tags: system

    - name: Install git
      yum:
        name: git
        state: present
      tags: system

    - name: Open required ports in firewall
      firewalld:
        port: "{{ item }}"
        permanent: yes
        state: enabled
      loop:
        - 80/tcp
        - 443/tcp
        - 22/tcp
        - 3000/tcp
        - 9090/tcp
        - 5000/tcp
      tags: firewall
```

🔍 Что происходит при запуске:

```
$ ansible-playbook -i inventory playbook.yml
```

Execution flow:

```
ANSIBLE (Local Machine)
  ↓
1. Читает playbook.yml
  ↓
2. Читает inventory (список серверов):
   [webservers]
   35.158.171.183 ansible_user=ec2-user ansible_ssh_private_key_file=my-devops-key.pem
  ↓
3. SSH подключение к 35.158.171.183
  ↓
═══════════════════════════════════════════════════════════════
AWS SERVER (35.158.171.183)
═══════════════════════════════════════════════════════════════
  ↓
4. TASK: Update system packages
   $ sudo yum update -y
   [▓▓▓
```

```
   9.4. Deploy with Docker Compose
        $ cd /opt/health-dashboard
        $ docker compose up -d --build
        changed: [35.158.171.183]
   ↓
   9.5. Wait for /health endpoint
        $ curl http://localhost:5000/health
        Retry 1/20 ... FAILED (connection refused)
        Retry 2/20 ... FAILED
        Retry 3/20 ... SUCCESS (200 OK)
        ok: [35.158.171.183]
  ↓
═══════════════════════════════════════════════════════════
PLAY RECAP
═══════════════════════════════════════════════════════════
35.158.171.183  : ok=15  changed=10  unreachable=0  failed=0
  ↓
✅ Deployment complete! (Total: ~8-10 min)
```

📋 Разбор roles/app/tasks/main.yml:

```yaml
- name: Create application directory
  file:
    path: "{{ app_dir }}"              # /opt/health-dashboard
    state: directory
    owner: ec2-user
    group: ec2-user
    mode: '0755'

- name: Clone or update application repository
  git:
    repo: "https://github.com/zaburdaev/my-devops-project.git"
    dest: "{{ app_dir }}"
    version: main
    force: true
  become_user: ec2-user

- name: Copy .env file to server
  copy:
    src: "{{ playbook_dir }}/../.env"
    dest: "{{ app_dir }}/.env"
    owner: ec2-user
    group: ec2-user
    mode: '0600'

- name: Deploy application with Docker Compose
  command: docker compose up -d --build
  args:
    chdir: "{{ app_dir }}"
  become_user: ec2-user

- name: Wait for application to be healthy
  uri:
    url: "http://localhost:5000/health"
    status_code: 200
  register: result
  until: result.status == 200
  retries: 20              # Попробует 20 раз
  delay: 10                # Каждые 10 секунд
```

🎯 Ansible Inventory:

```ini
[webservers]
35.158.171.183 ansible_user=ec2-user ansible_ssh_private_key_file=./my-devops-key.pem

[webservers:vars]
ansible_python_interpreter=/usr/bin/python3
```

Inventory = список серверов которыми управляет Ansible

📊 Ansible vs Manual SSH:

```
╔═══════════════════════════════════════════════════════════╗
║                  MANUAL SSH (OLD WAY) ❌                  ╒
╟═══════════════════════════════════════════════════════════╢
│  ssh ec2-user@35.158.171.183                              │
│  sudo yum update -y                                       │
│  sudo yum install -y docker git                           │
│  sudo systemctl start docker                              │
│  curl -L ... -o /usr/local/bin/docker-compose             │
│  chmod +x /usr/local/bin/docker-compose                   │
│  git clone https://github.com/zaburdaev/...               │
│  cd /opt/health-dashboard                                 │
│  docker compose up -d                                     │
│                                                           │
│  ПРОБЛЕМЫ:                                                │
│  ❌ Нужно помнить все команды                             │
│  ❌ Можно забыть шаг                                      │
│  ❌ Не reproducible (сложно повторить на новом сервере)   │
│  ❌ Нельзя автоматизировать                               │
│  ❌ Нет логов                                             │
╚═══════════════════════════════════════════════════════════╝

╔═══════════════════════════════════════════════════════════╗
║                  ANSIBLE (NEW WAY) ✅                     ╒
╟═══════════════════════════════════════════════════════════╢
│  ansible-playbook -i inventory playbook.yml               │
│                                                           │
│  ПРЕИМУЩЕСТВА:                                            │
│  ✅ Одна команда для всего                                │
│  ✅ Idempotent (можно запускать много раз безопасно)      │
│  ✅ Reproducible (работает на любом сервере)              │
│  ✅ Можно версионировать playbook в Git                   │
│  ✅ Автоматические логи и отчёты                          │
│  ✅ Можно управлять 100 серверами одновременно            │
╚═══════════════════════════════════════════════════════════╝
```

🎯 Что сказать на защите:

> "Я использовал Ansible для автоматизации настройки AWS сервера.
>
> Ansible playbook выполняет:
> - Обновление системных пакетов
> - Установку Docker, Docker Compose, Git
> - Настройку firewall (открывает нужные порты)
> - Клонирование репозитория
> - Запуск приложения через docker compose
> - Health check (ждёт пока приложение стартует)
>
> Всё это одна команда: `ansible-playbook -i inventory playbook.yml`.
>
> Ansible idempotent - можно запускать много раз, он применит только нужные изменения.
>
> Playbook описан в YAML формате и хранится в Git - это Configuration as Code.
>
> Если нужно настроить 10 серверов - достаточно добавить их в inventory, Ansible настроит все параллельно."

📋 Таблица: Что делает Ansible

Task	Команда	Время	Idempotent?
Update packages	`yum update -y`	2 min	✅ Yes
Install Git	`yum install git`	30 sec	✅ Yes (skip if exists)
Install Docker	`yum install docker`	1 min	✅ Yes
Clone repo	`git clone ...`	20 sec	✅ Yes (pull if exists)
Docker compose up	`docker compose up -d`	3 min	✅ Yes (restart changed)
TOTAL		**~8 min**	✅

---

6. Kubernetes (Container Orchestration)

📁 Файлы: `k8s/*.yaml`, `k8s/helm/`

🎯 Что это?

Kubernetes (K8s) - это система оркестрации контейнеров. Она управляет запуском, масштабированием, и мониторингом контейнеров в production.

🏗️ Kubernetes манифесты:

```
k8s/
├── namespace.yaml          # Изолированное пространство
├── configmap.yaml          # Конфигурация (не секретная)
├── secret.yaml             # Секреты (пароли, ключи)
├── deployment.yaml         # Deployment (запуск pods)
├── service.yaml            # Service (LoadBalancer)
└── helm/                   # Helm Chart (пакетный менеджер)
```

🔍 Разбор каждого манифеста:

1. Namespace

```yaml
apiVersion: v1
kind: Namespace
metadata:
  name: health-dashboard
```

Что делает:

```
kubectl apply -f namespace.yaml
  ↓
Kubernetes создаёт namespace "health-dashboard"
  ↓
Все ресурсы будут в этом namespace (изоляция)
```

Namespace = папка для ресурсов (pods, services, deployments)

---

2. ConfigMap

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: health-dashboard-config
  namespace: health-dashboard
data:
  FLASK_ENV: "production"
  APP_PORT: "5000"
  POSTGRES_HOST: "postgres"
  REDIS_HOST: "redis"
```

Что делает:

```
kubectl apply -f configmap.yaml
  ↓
Kubernetes сохраняет конфигурацию
  ↓
Pods могут читать эти переменные:
  - FLASK_ENV=production
  - APP_PORT=5000
  - ...
```

ConfigMap = хранилище non-sensitive конфигурации

---

3. Secret

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: health-dashboard-secret
  namespace: health-dashboard
type: Opaque
data:
  DATABASE_URL: cG9zdGdyZXNxbDovL2hlYWx0aHVzZXI6aGVhbHRocGFzc...  # base64
```

Что делает:

```
kubectl apply -f secret.yaml
  ↓
Kubernetes сохраняет секрет (encrypted at rest)
  ↓
Pods могут читать как environment variables
```

Secret = хранилище sensitive данных (пароли, токены)

---

4. Deployment

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: health-dashboard
  namespace: health-dashboard
spec:
  replicas: 2                          # Запустить 2 pods
  selector:
    matchLabels:
      app: health-dashboard
  template:
    metadata:
      labels:
        app: health-dashboard
    spec:
      containers:
        - name: health-dashboard
          image: oskalibriya/health-dashboard:latest
          ports:
            - containerPort: 5000
          envFrom:
            - configMapRef:
                name: health-dashboard-config
            - secretRef:
                name: health-dashboard-secret
          resources:
            requests:
              memory: "128Mi"
              cpu: "100m"
            limits:
              memory: "256Mi"
              cpu: "250m"
          livenessProbe:
            httpGet:
              path: /health
              port: 5000
            initialDelaySeconds: 15
            periodSeconds: 30
          readinessProbe:
            httpGet:
              path: /health
              port: 5000
            initialDelaySeconds: 5
            periodSeconds: 10
```

Что происходит:

```
kubectl apply -f deployment.yaml
  ↓
Kubernetes создаёт Deployment "health-dashboard"
  ↓
Deployment создаёт ReplicaSet
  ↓
ReplicaSet создаёт 2 Pods:
  - health-dashboard-7d8f9c5-abc12
  - health-dashboard-7d8f9c5-def34
  ↓
Каждый Pod запускает контейнер:
  docker run oskalibriya/health-dashboard:latest
  ↓
Kubernetes инжектирует env vars из ConfigMap и Secret
  ↓
Каждые 30 секунд Kubernetes проверяет liveness:
  curl http://pod-ip:5000/health
  ✅ OK → Pod healthy
  ❌ FAIL 3 раза → Kubernetes kills pod и создаёт новый
  ↓
Каждые 10 секунд Kubernetes проверяет readiness:
  curl http://pod-ip:5000/health
  ✅ OK → Pod gets traffic
  ❌ FAIL → Pod removed from Service (no traffic)
```

---

5. Service (LoadBalancer)

```yaml
apiVersion: v1
kind: Service
metadata:
  name: health-dashboard-service
  namespace: health-dashboard
spec:
  type: LoadBalancer
  selector:
    app: health-dashboard
  ports:
    - name: http
      protocol: TCP
      port: 80
      targetPort: 5000
```

Что происходит:

```
kubectl apply -f service.yaml
  ↓
Kubernetes создаёт Service "health-dashboard-service"
  ↓
Service находит все Pods с label "app=health-dashboard"
  - health-dashboard-7d8f9c5-abc12 (10.1.1.5:5000)
  - health-dashboard-7d8f9c5-def34 (10.1.1.6:5000)
  ↓
Service создаёт LoadBalancer (cloud provider):
  External IP: 3.127.155.114
  ↓
USER → http://3.127.155.114/health
  ↓
LoadBalancer распределяет на pods (round-robin):
  ├─ 50% → Pod 1 (10.1.1.5:5000)
  └─ 50% → Pod 2 (10.1.1.6:5000)
  ↓
RESPONSE ← {"status": "healthy"}
```

Service = load balancer + service discovery

🔄 Kubernetes Architecture:

```
┌─────────────────────────────────────────────────────────────┐
│                    KUBERNETES CLUSTER                       │
├─────────────────────────────────────────────────────────────┤
│                                                             │
│   ┌─────────────────────────────────────────────────────┐   │
│   │  NAMESPACE: health-dashboard                        │   │
│   ├─────────────────────────────────────────────────────┤   │
│   │                                                     │   │
│   │   ┌─────────────────────────────────────────────┐   │   │
│   │   │  DEPLOYMENT: health-dashboard (replicas=2)  │   │   │
│   │   ├─────────────────────────────────────────────┤   │   │
│   │   │                                             │   │   │
│   │   │  POD 1: health-dashboard-7d8f9c5-abc12      │   │   │
│   │   │  ├─ Container: health-dashboard:latest      │   │   │
│   │   │  ├─ IP: 10.1.1.5                            │   │   │
│   │   │  ├─ Port: 5000                              │   │   │
│   │   │  ├─ CPU: 100m (request), 250m (limit)       │   │   │
│   │   │  ├─ Memory: 128Mi (request), 256Mi (limit)  │   │   │
│   │   │  ├─ Liveness: curl /health every 30s        │   │   │
│   │   │  └─ Readiness: curl /health every 10s       │   │   │
│   │   │                                             │   │   │
│   │   │  POD 2: health-dashboard-7d8f9c5-def34      │   │   │
│   │   │  ├─ Container: health-dashboard:latest      │   │   │
│   │   │  ├─ IP: 10.1.1.6                            │   │   │
│   │   │  └─ ... (same as Pod 1)                     │   │   │
│   │   │                                             │   │   │
│   │   └─────────────────────────────────────────────┘   │   │
│   │                         ↑                           │   │
│   │                         │                           │   │
│   │   ┌─────────────────────────────────────────────┐   │   │
│   │   │  SERVICE: health-dashboard-service          │   │   │
│   │   │  Type: LoadBalancer                         │   │   │
│   │   │  External IP: 3.127.155.114                 │   │   │
│   │   │  Port: 80 → targetPort: 5000                │   │   │
│   │   │                                             │   │   │
│   │   │  Backends:                                  │   │   │
│   │   │  ├─ 10.1.1.5:5000 (Pod 1)                   │   │   │
│   │   │  └─ 10.1.1.6:5000 (Pod 2)                   │   │   │
│   │   └─────────────────────────────────────────────┘   │   │
│   │                                                     │   │
│   │   ┌─────────────────────────────────────────────┐   │   │
│   │   │  CONFIGMAP: health-dashboard-config         │   │   │
│   │   │  - FLASK_ENV=production                     │   │   │
│   │   │  - APP_PORT=5000                            │   │   │
│   │   └─────────────────────────────────────────────┘   │   │
│   │                                                     │   │
│   │   ┌─────────────────────────────────────────────┐   │   │
│   │   │  SECRET: health-dashboard-secret            │   │   │
│   │   │  - DATABASE_URL (encrypted)                 │   │   │
│   │   └─────────────────────────────────────────────┘   │   │
│   │                                                     │   │
│   └─────────────────────────────────────────────────────┘   │
│                                                             │
└─────────────────────────────────────────────────────────────┘
```

🎯 Что сказать на защите:

> "В проекте есть Kubernetes манифесты для production-ready deployment.
>
> Kubernetes Deployment запускает 2 replica pods для high availability. Если один pod падает, Kubernetes автоматически создаёт новый.
>
> Использую liveness probe для проверки здоровья (каждые 30 сек). Если pod не отвечает 3 раза подряд - Kubernetes его перезапускает.
>
> Использую readiness probe для определения когда pod готов принимать трафик.
>
> Service type LoadBalancer создаёт external IP и распределяет трафик между pods.
>
> Конфигурация хранится в ConfigMap, секреты (пароли) в Secret (encrypted at rest).
>
> Всё развёртывается командой: `kubectl apply -f k8s/`.
>
> Также есть Helm Chart - это пакетный менеджер для Kubernetes, позволяет версионировать deployment и легко обновлять параметры (например, количество replicas)."

---

7. Monitoring (Prometheus + Grafana)

📁 Файлы: `monitoring/prometheus.yml`, `monitoring/grafana/`

🎯 Что это?

Мониторинг - это система сбора метрик и визуализации состояния приложения.

🏗️ Архитектура мониторинга:

```
┌──────────────────────────────────────────────────────────────┐
│                      MONITORING STACK                        │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│  1. FLASK APP (port 5000)                                    │
│     └─ Endpoint: /metrics (Prometheus format)                │
│        ├─ app_request_total                                  │
│        ├─ app_request_latency_seconds                        │
│        ├─ system_cpu_usage_percent                           │
│        ├─ system_memory_usage_percent                        │
│        └─ system_disk_usage_percent                          │
│                                                              │
│  2. PROMETHEUS (port 9090)                                   │
│     ├─ Scrapes /metrics every 60 seconds                     │
│     ├─ Stores in TSDB (Time Series Database)                 │
│     ├─ Retention: 3 hours                                    │
│     └─ Query language: PromQL                                │
│                                                              │
│  3. GRAFANA (port 3000)                                      │
│     ├─ Reads data from Prometheus                            │
│     ├─ Visualizes as dashboards                              │
│     ├─ Pre-configured dashboard: Health Dashboard Metrics    │
│     └─ Login: admin / admin                                  │
│                                                              │
└──────────────────────────────────────────────────────────────┘
```

🔍 Prometheus Configuration:

```yaml
global:
  scrape_interval: 60s              # Собирать метрики каждые 60 сек
  evaluation_interval: 60s          # Проверять alerts каждые 60 сек

scrape_configs:
  - job_name: 'prometheus'          # Мониторит сам себя
    static_configs:
      - targets: ['localhost:9090']

  - job_name: 'flask-app'           # Мониторит Flask приложение
    static_configs:
      - targets: ['app:5000']       # Docker service name
```

Что происходит:

```
PROMETHEUS SCRAPER (каждые 60 сек)
  ↓
1. curl http://app:5000/metrics
  ↓
RESPONSE:
# TYPE app_request_total counter
app_request_total{method="GET",endpoint="/health",http_status="200"} 1523.0
app_request_total{method="GET",endpoint="/api/system-info",http_status="200"} 89.0

# TYPE app_request_latency_seconds histogram
app_request_latency_seconds_bucket{endpoint="/health",le="0.005"} 1200.0
app_request_latency_seconds_bucket{endpoint="/health",le="0.01"} 1500.0
...

# TYPE system_cpu_usage_percent gauge
system_cpu_usage_percent 24.5

# TYPE system_memory_usage_percent gauge
system_memory_usage_percent 62.3

# TYPE system_disk_usage_percent gauge
system_disk_usage_percent 45.1
  ↓
2. Prometheus парсит метрики
  ↓
3. Сохраняет в TSDB:

   Timestamp          | Metric                    | Value
   -------------------|---------------------------|-------
   2026-04-28 10:00:00| system_cpu_usage_percent | 24.5
   2026-04-28 10:01:00| system_cpu_usage_percent | 26.3
   2026-04-28 10:02:00| system_cpu_usage_percent | 23.1
   ...
  ↓
4. Можно query через PromQL:

   Query: rate(app_request_total[5m])
   Result: 2.5 requests/second

   Query: avg_over_time(system_cpu_usage_percent[1h])
   Result: 25.3% (average CPU за последний час)
```

📊 Grafana Dashboards:

Provisioned datasource (monitoring/grafana/provisioning/datasources/datasources.yaml):

```yaml
apiVersion: 1
datasources:
  - name: Prometheus
    type: prometheus
    access: proxy
    url: http://prometheus:9090
    isDefault: true
```

Dashboard includes:

```
HEALTH DASHBOARD METRICS
├─ Panel 1: CPU Usage (%)
│  └─ Query: system_cpu_usage_percent
│     Visualization: Line chart
│
├─ Panel 2: Memory Usage (%)
│  └─ Query: system_memory_usage_percent
│     Visualization: Line chart
│
├─ Panel 3: Disk Usage (%)
│  └─ Query: system_disk_usage_percent
│     Visualization: Gauge
│
├─ Panel 4: Request Rate (req/s)
│  └─ Query: rate(app_request_total[5m])
│     Visualization: Graph
│
├─ Panel 5: Response Time (ms)
│  └─ Query: histogram_quantile(0.95, app_request_latency_seconds_bucket)
│     Visualization: Graph (95th percentile)
│
└─ Panel 6: Container Health
   └─ Query: up{job="flask-app"}
      Visualization: Stat (1=UP, 0=DOWN)
```

🔄 Monitoring Flow:

```
╔═══════════════════════════════════════════════════════════╗
║                      MONITORING FLOW                      ║
╞═══════════════════════════════════════════════════════════╡
│                                                           │
│  USER makes request                                       │
│    ↓                                                      │
│  Flask App handles request                                │
│    ├─ REQUEST_COUNT.inc()                                 │
│    ├─ REQUEST_LATENCY.observe(0.015)                      │
│    ├─ SYSTEM_CPU.set(25.3)                                │
│    ├─ SYSTEM_MEMORY.set(62.1)                             │
│    └─ SYSTEM_DISK.set(45.0)                               │
│    ↓                                                      │
│  Metrics available at /metrics endpoint                   │
│    ↓                                                      │
│  Prometheus scrapes /metrics (every 60s)                  │
│    └─ Stores in TSDB                                      │
│    ↓                                                      │
│  Grafana queries Prometheus                               │
│    └─ Renders dashboard                                   │
│    ↓                                                      │
│  USER views http://IP:3000                                │
│    └─ Sees real-time metrics                              │
│                                                           │
╒═══════════════════════════════════════════════════════════╒
```

🎯 Что сказать на защите:

> "Я настроил full-stack monitoring с Prometheus и Grafana.
>
> **Prometheus:**
> - Scrapes метрики из Flask приложения каждые 60 секунд
> - Собирает: CPU usage, memory usage, disk usage, request count, response time
> - Хранит данные в TSDB (Time Series Database) с retention 3 часа
>
> **Grafana:**
> - Подключается к Prometheus как data source
> - Автоматически provisioned (не нужно настраивать вручную)
> - Dashboard показывает 6 panels: CPU, Memory, Disk, Request Rate, Response Time, Container Health
> - Обновляется в real-time
>
> **Flask Application:**
> - Использую библиотеку `prometheus_client`
> - Экспортирую метрики через `/metrics` endpoint
> - Counter для запросов, Histogram для latency, Gauge для system metrics
>
> Весь мониторинг запускается автоматически через `docker-compose.yml`. Grafana доступна на порту 3000 (admin/admin)."

---

8. Testing (Unit Tests)

📁 Файлы: `tests/test_app.py`, `tests/conftest.py`

🎯 Что это?

Unit tests - это автоматические тесты которые проверяют что код работает правильно.

🏗️ Структура тестов:

```
tests/
├── __init__.py              # Empty file (makes it a package)
├── conftest.py              # Pytest fixtures
└── test_app.py              # Unit tests
```

📋 Разбор test_app.py:

```python
def test_health_endpoint_returns_200(client):
    """Test that /health returns HTTP 200."""
    response = client.get("/health")
    assert response.status_code == 200

def test_health_response_structure(client):
    """Test that /health returns the correct JSON structure."""
    response = client.get("/health")
    data = json.loads(response.data)
    assert "status" in data
    assert "timestamp" in data
    assert "uptime_seconds" in data
    assert data["status"] == "healthy"

def test_health_status_is_healthy(client):
    """Test that /health reports a healthy status."""
    response = client.get("/health")
    data = json.loads(response.data)
    assert data["status"] == "healthy"
```

🔄 Как запускаются тесты:

```
$ pytest tests/ -v --cov=app
```

Output:

```
============ test session starts ============
platform linux -- Python 3.11.0
collected 12 items

tests/test_app.py::test_index_returns_200 PASSED                    [  8%]
tests/test_app.py::test_index_contains_html PASSED                  [ 16%]
tests/test_app.py::test_health_endpoint_returns_200 PASSED          [ 25%]
tests/test_app.py::test_health_response_structure PASSED            [ 33%]
tests/test_app.py::test_health_status_is_healthy PASSED             [ 41%]
tests/test_app.py::test_system_info_returns_200 PASSED              [ 50%]
tests/test_app.py::test_system_info_json_structure PASSED           [ 58%]
tests/test_app.py::test_metrics_endpoint_returns_200 PASSED         [ 66%]
tests/test_app.py::test_metrics_prometheus_format PASSED            [ 75%]
tests/test_app.py::test_404_error PASSED                            [ 83%]
tests/test_app.py::test_request_counter_increments PASSED           [ 91%]
tests/test_app.py::test_system_gauges_populated PASSED              [100%]

----------- coverage: platform linux -----------
Name                 Stmts   Miss  Cover   Missing
--------------------------------------------------
app/__init__.py          2      0   100%
app/app.py             183     27    85%   89-92, 145-148, ...
--------------------------------------------------
TOTAL                  185     27    85%

============ 12 passed in 2.34s ============
```

🎯 Что сказать на защите:

> "Написал 12 unit tests с использованием pytest.
>
> Тесты проверяют:
> - Health endpoint возвращает 200 и правильный JSON
> - System info endpoint работает
> - Prometheus metrics endpoint возвращает правильный формат
> - Error handling (404)
>
> Code coverage: 85%**
>
> Тесты запускаются автоматически в CI/CD pipeline. Если хотя бы один тест fails - deployment не происходит."

---

9. Nginx (Reverse Proxy)

📁 Файл: `nginx/nginx.conf`

🎯 Что это?

Nginx - это web server и reverse proxy. Он принимает все HTTP запросы и проксирует их на Flask приложение.

📋 Nginx Configuration:

```
upstream flask_app {
    server app:5000;              # Backend: Flask app
}

server {
    listen 80;                     # Слушает порт 80
    server_name _;                 # Принимает любой hostname

    # Security headers
    add_header X-Frame-Options "SAMEORIGIN" always;
    add_header X-Content-Type-Options "nosniff" always;
    add_header X-XSS-Protection "1; mode=block" always;

    location / {
        proxy_pass http://flask_app;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }

    location /metrics {
        proxy_pass http://flask_app/metrics;
    }

    location /nginx-health {
        return 200 'OK';
    }
}
```

🔄 Request Flow:

```
USER → http://35.158.171.183/health
  ↓
AWS Security Group (port 80 open)
  ↓
EC2 Instance (port 80)
  ↓
Docker: nginx container (port 80)
  ↓
nginx.conf: proxy_pass http://flask_app
  ↓
Docker network: app:5000
  ↓
Flask App (port 5000)
  ↓
Response: {"status": "healthy"}
  ↓
USER ← HTTP 200 {"status": "healthy"}
```

🎯 Что сказать на защите:

> “Использую Nginx как reverse proxy перед Flask приложением.
>
> Nginx:
> - Принимает все HTTP запросы на порту 80
> - Проксирует на Flask (port 5000)
> - Добавляет security headers (X-Frame-Options, X-XSS-Protection)
> - Может масштабироваться (load balancing на несколько Flask инстансов)
>
> Это best practice для production - не выставлять Flask напрямую в интернет.”

✅ Итого: Все этапы

```
┌──────────────────────────────────────────────────────────────┐
│              ПОЛНЫЙ DEVOPS ЖИЗНЕННЫЙ ЦИКЛ                    │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│  1. FLASK APP - Приложение (Python)                          │
│  2. DOCKER - Контейнеризация                                 │
│  3. DOCKER COMPOSE - Локальная оркестрация (7 сервисов)      │
│  4. CI/CD - GitHub Actions (Test → Build → Deploy)           │
│  5. TERRAFORM - Infrastructure as Code (AWS)                 │
│  6. ANSIBLE - Configuration Management                       │
│  7. KUBERNETES - Production orchestration                    │
│  8. MONITORING - Prometheus + Grafana                        │
│  9. TESTING - Unit tests (pytest)                            │
│ 10. NGINX - Reverse proxy                                    │
│                                                              │
│  ✅ Весь стек DevOps в одном проекте!                        │
│                                                              │
└──────────────────────────────────────────────────────────────┘
```

Теперь ты можешь уверенно рассказать про каждый этап на защите! 💪🚀